Priority Stack

(1 second, 64 megabytes)

มี Data Structure หนึ่ง ที่ชื่อว่า Priority Stack สามารถทำคำสั่ง ได้ดังนี้

- push(x) — ดัน x เข้าบน Stack
- pop() — นำตัวด้านบน Stack ออก โดยหาก Stack ว่าง ก็จะข้ามไป
- priority_pop(x) — นำตัวที่มีค่า x ออกจาก Stack โดยหาก Stack ไม่มีสมาชิกที่มีค่า x ก็จะข้ามไป

จงจำลองการทำงานของ Priority Stack ดังนี้

ข้อมูลนำเข้า

บรรทัดแรก ระบุจำนวนเต็ม 1 จำนวน q ($1 \le q \le 10^5$) — แทนจำนวนครั้งการทำคำสั่ง
จากนั้นตามด้วย q บรรทัด แต่ละบรรทัด จะอยู่ในรูปแบบหนึ่ง จากต่อไปนี้

- 1 x — แทนการทำ push(x)
- 2 — แทนการทำ pop(x)
- 3 x — แทนการทำ priority_pop(x)

รับประกันว่า จะไม่มีจังหวะไหน ที่มีสมาชิกค่าซ้ำกันอยู่ใน Stack เสมอ

ข้อมูลนำออก

สำหรับแต่ละคำสั่ง แบบ pop() และ priority_pop() ให้ส่งออกสมาชิกที่ pop ออก หากไม่มีสมาชิกที่ตรงเงื่อนไข สำหรับการ pop (ถูกข้าม) ให้ส่งออก -1

บรรทัดสุดท้าย แสดงสมาชิกใน Stack ที่เหลือในตอนสุดท้าย จากตัวบนสุดไปล่างสุด

ปัญหาย่อย

1. (20 คะแนน) ไม่มีการใช้ priority_pop
2. (40 คะแนน) $q \le 5 \cdot 10^3$
3. (40 คะแนน) ไม่มีเงื่อนไขเพิ่มเติม

ตัวอย่างข้อมูลนำเข้าส่งออก

ข้อมูลนำเข้า	ข้อมูลนำออก
9 2 1 5 1 9 3 4 1 4 2 1 7 3 5 1 6	-1 -1 4 5 6 7 9

คำอธิบาย

ในตัวอย่าง มีการทำคำสั่งบน Stack ทั้งหมด 9 คำสั่ง ดังนี้

- pop() — เนื่องจากไม่มีสมาชิกใน Stack จึงถูกข้าม และส่งออก -1
- push(5) — Stack จากบนลงล่างเป็น {5}
- push(9) — Stack จากบนลงล่างเป็น {9,5}
- priority_pop(4) — เนื่องจากไม่มีสมาชิกที่ค่า 4 ใน Stack จึงถูกข้าม และส่งออก -1
- push(4) — Stack จากบนลงล่างเป็น {4,9,5}
- pop() — Stack จากบนลงล่างเป็น {9,5}
- push(7) — Stack จากบนลงล่างเป็น {7,9,5}
- priority_pop(5) — Stack จากบนลงล่างเป็น {7,9}
- push(6) — Stack จากบนลงล่างเป็น {6,7,9}

ทำให้ Stack ในตอนสุดท้าย มีสมาชิกเป็น {6,7,9} จากบนลงล่าง